

임베디드 소프트웨어

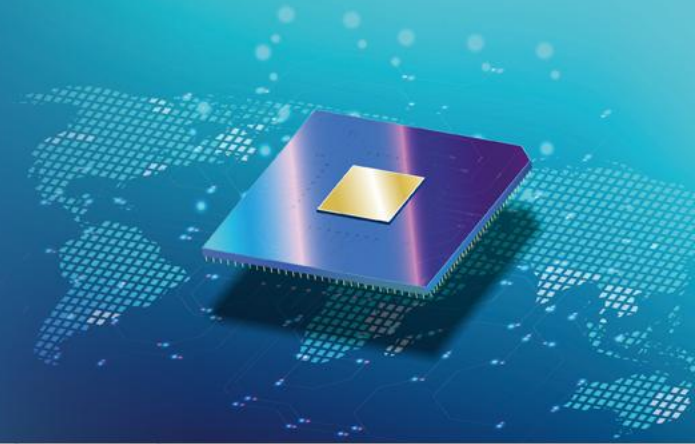
서울대학교 차세대반도체 혁신융합대학





CHAPTER 1

수업 소개



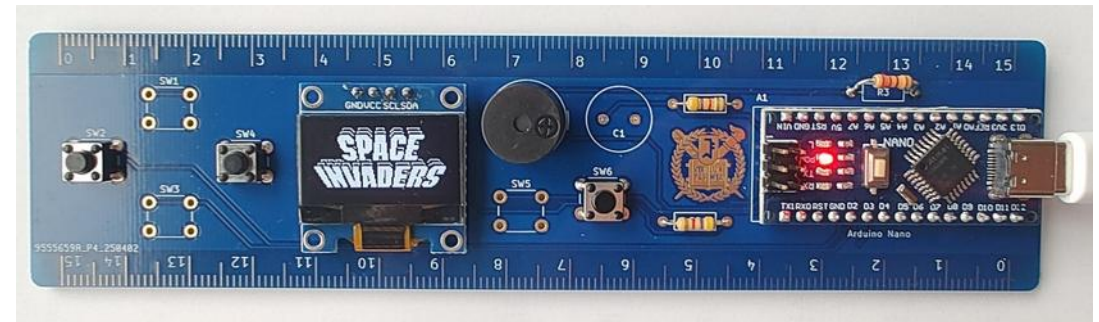
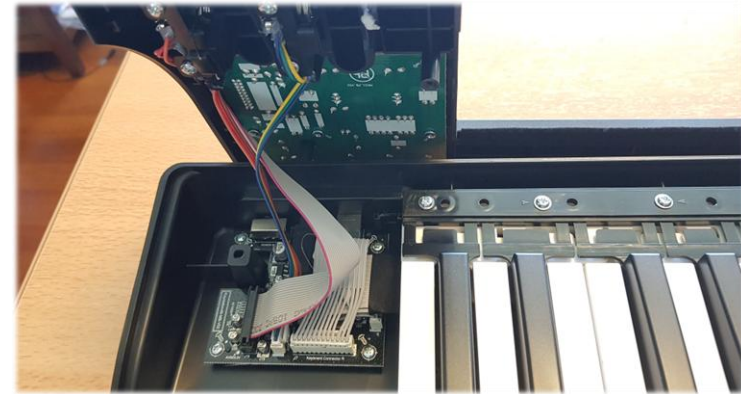
강사 소개

이우빈 교수

- 서울대학교 전자공학과 83학번
- 미국 University of Washington 석사, 박사
- 미국 반도체 스타트업 회사와 대기업 근무
- SK 하이닉스 임원
- 영상 처리 시스템 반도체의 개발, 기획, 마케팅, 영업
- 서울대학교 차세대반도체 혁신융합대학 교수



- 임베디드 시스템 관련 수업 개발 및 강의
- MCU 응용 경진대회, 대한민국 과학기술 축제, SEDEX에서 반도체 체험 행사 운영



임베디드 시스템의 예

Embedded = 내장된, 내재된



수업의 내용과 목표

- 마이크로 컨트롤러 (MCU)의 하드웨어 레지스터를 직접 제어하는 소프트웨어를 작성하여, I/O 핀에 연결된 하드웨어 (LED, Display 등)을 동작해 봄.
- 화려한 응용 프로그램 보다는 기본 원리에 집중.
- 수업목표: 소프트웨어가 어떻게 하드웨어를 제어하여 임베디드 시스템이 동작하는지 확실히 이해.

교과목 이름은 소프트웨어이나, 하드웨어를 이해하여야 소프트웨어를 작성할 수 있음.

- Arduino 경험이 있으면 도움이 되나, 필수는 아님.
- 기초 C 프로그래밍은 할 수 있어야 함.
 - for, while 문
 - 변수의 종류, 배열, 포인터
 - 양수와 음수의 표현
 - Bitwise 논리 연산 (&, |, ~)

- Breadboard (빵판)에 IC 부품을 꽂아 회로를 구성함.
- Breadboard 회로 작성 경험이 없으면 도움 요청.

수업 방식

- 1~1.5시간의 강의
 - 연결할 하드웨어 부품의 동작 원리
 - 부분 별 MCU 하드웨어의 동작 원리 및 Register 설명
 - 작성해야 할 코드의 기본 구조
- 1.5~2시간의 실습.
 - 각자 회로를 연결하고 코드를 작성하여 동작을 확인.
 - 2주마다 과제 (수업 내용의 구현).
- 5주마다 프로젝트 수행.
 - 1차 프로젝트 (1~4회 수업 내용 응용)
 - 2차 프로젝트 (6~9회 수업 내용 응용)
 - 3차 프로젝트 (11~14회 수업 내용 응용 + 추가 기능)
 - 제출 결과물은 회로도, 코드, 동작 동영상.

Arduino를 사용하지 않는 이유

하드웨어를 추상화 (abstraction) 하여 개발자에게 보이지 않게 함.

다른 사람이 작성한 부품별 Device Driver 라이브러리를 설치하면 하드웨어의 이해 없이도 코드 작성 및 동작 가능.

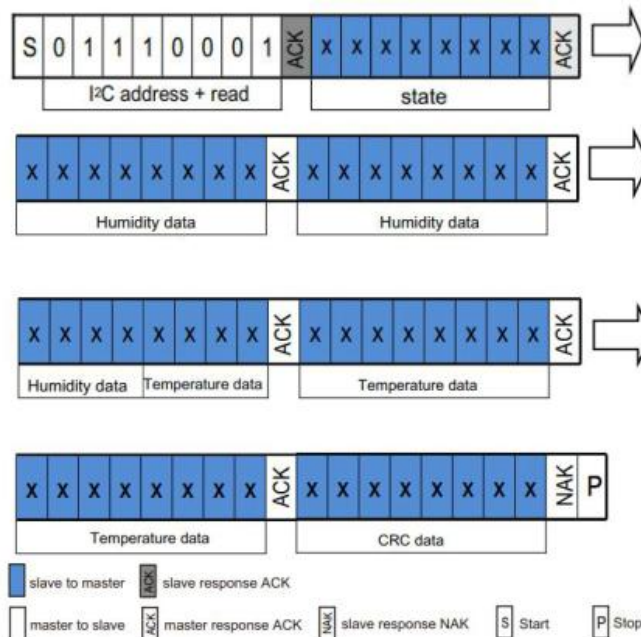
예시: AHT20 온도센서 (I²C Interface)

Arduino Library 활용

```
void loop() {  
  if(aht20.startMeasurementReady(true)) {  
    Serial.print(aht20.getTemperature_C());  
    Serial.print(" C, ");  
    // Get temp in Fahrenheit (F)  
    Serial.print(aht20.getTemperature_F());  
    Serial.print(" F\t");  
    Serial.print(aht20.getHumidity_RH());  
    Serial.println(" %RH");  
    delay(5000);  
  }  
}
```

- 장점
 - Hardware Independent Code
 - 응용 프로그램을 쉽게 작성.
- 단점: 하드웨어를 이해할 수 있는 기회가 없어짐.

수업 방식: Datasheet을 분석하여 직접 Device Driver 코드를 작성



사용할 하드웨어와 소프트웨어



Microchip ATtiny4313

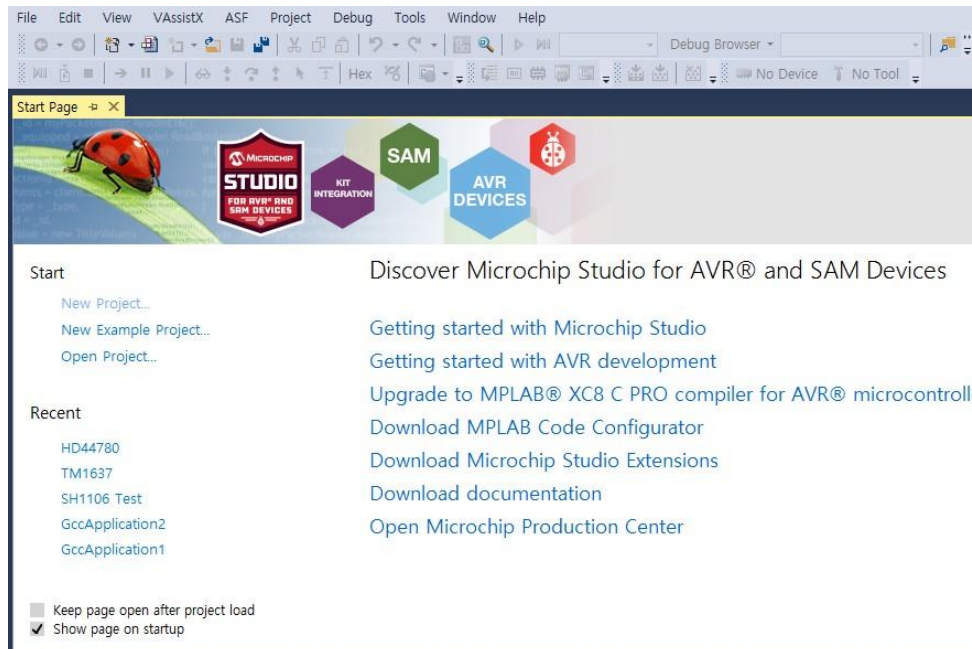
- 8-bit, 20MHz
- 4k bytes Flash Memory
- 256 bytes SRAM



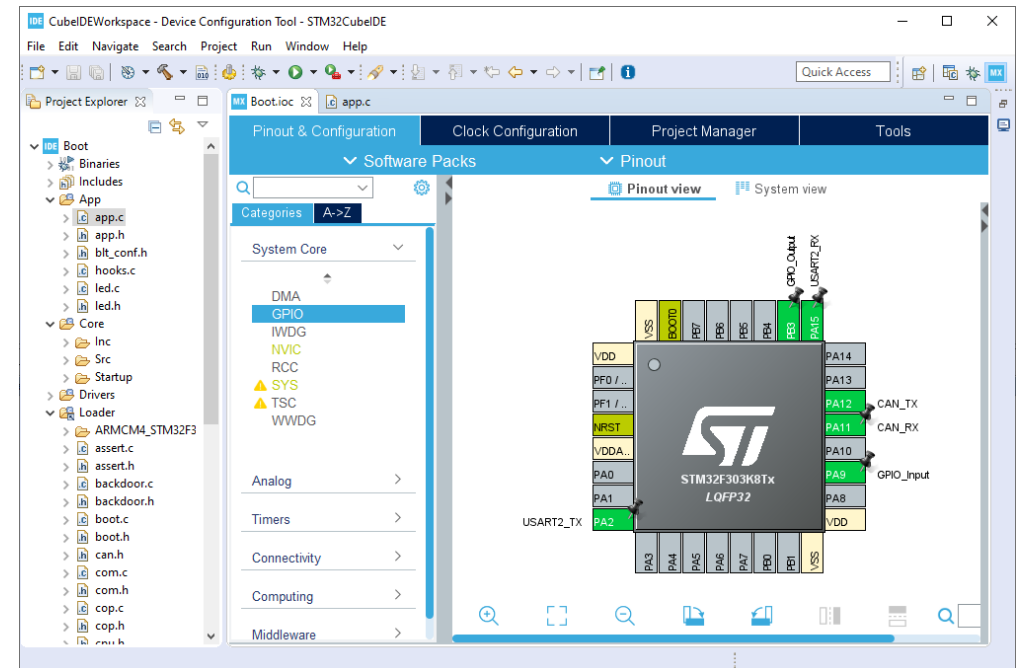
ST Micro STM32F103 (Bluepill) 개발 보드

- 32-bit, 72MHz
- 64k bytes Flash Memory
- 20k bytes SRAM

Microchip Studio



STM32CubeIDE



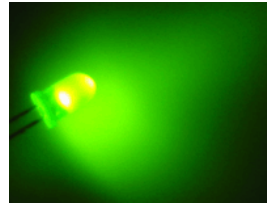
1~5회차 수업 구성



1회



Microchip Studio 사용법



LED Blink

2회



Pushbutton Switch



문자 LCD display
HD44780, 병렬 인터페이스

3회



8-digit 7-segment display
MAX7219, SPI

4회

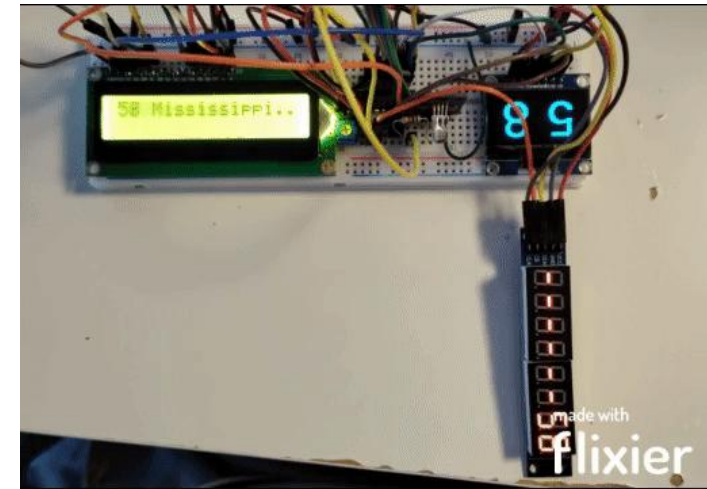


128x64 Graphics
OLED display
SSD1306, SPI



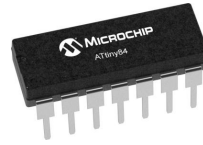
Shift Register IC
74HC595

5회



1~4주 학습 내용을 응용한 프로젝트

6~9회차 수업 구성

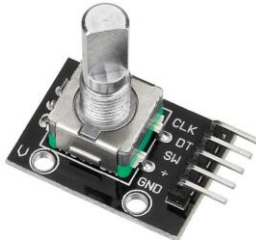


6회

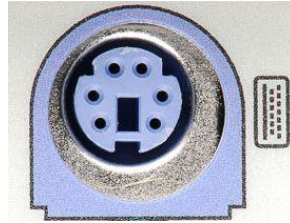
```
ISR (TIM0_OVF_vect) {
    // Do something here.
}

int main(void) {
    while (1);
}
```

Interrupt

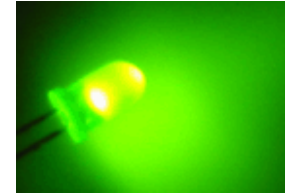


Rotary Encoder

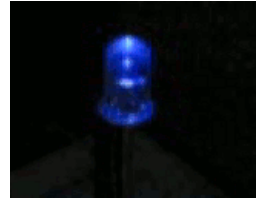


PS2 Keyboard

9회

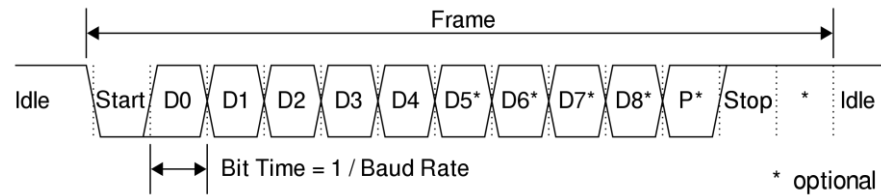


Timer

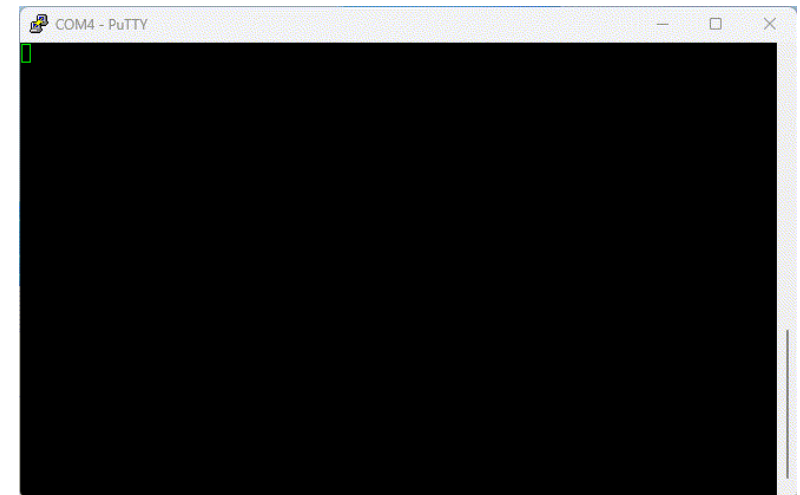


PWM

7회

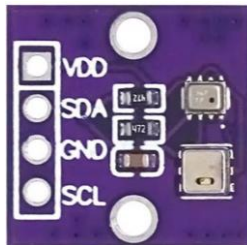


10회

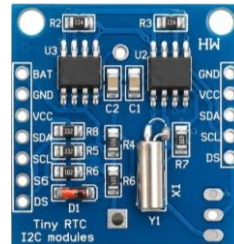


6~8주차의 학습 내용을 응용한 프로젝트

8회

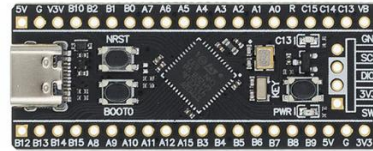


Temperature &
Pressure Sensor,
I²C



Real-Time Clock
& EEPROM, I²C

11~15주차 수업 구성



11회



STM32 사용법

STM32 SPI

14회



Animation

DMA

12회



Addressable LED

Rotary Encoder

13회



Color LCD (ILI9341)

15회

11~14주차의 학습 내용을 응용한 프로젝트

Datasheet의 중요성

- 각 부품의 외형적, 전기적, 기능적 특성을 설명한 기술 문서.
- 그 부품을 이해하기 위해서는 항상 datasheet 에서 출발.
- 모든 답은 datasheet에 있음.
- 부품에 따라서 1페이지 datasheet 부터 수백 페이지의 datasheet 가 있음.
- 추가 설명을 위한 Reference Manual, Application Note 등도 있음.



이 수업의 교과서는 Datasheet.

2026년 2학기 수업 계획

ATtiny4313 MCU

	날짜	수업 내용
1회	9월 4일	Microchip Studio 사용법, LED blink
2회	9월 11일	문자 LCD display (과제)
3회	9월 18일	SPI, 7-Segment LED display
4회	9월 25일*	OLED display, Shift Register
5회	10월 2일	1차 프로젝트

	날짜	수업 내용
6회	10월 9일*	Interrupt, Rotary Encoder, Keyboard
7회	10월 16일	UART (과제)
8회	10월 23일	Timer, PWM
9회	10월 30일	I ² C
10회	11월 6일	2차 프로젝트

STM32 MCU

	날짜	수업 내용
11회	11월 13일	STM32CubeIDE 사용법
12회	11월 20일*	Addressable LED (과제)
13회	11월 27일	Color LCD Controller
14회	12월 4일	Animation, DMA
15회	12월 11일	3차 프로젝트

* 공휴일 또는 휴강이라 보강 필요

평가 기준

배점 구성

평가 항목	배점
과제 (10% x 3)	30%
1차 프로젝트	20%
2차 프로젝트	20%
3차 프로젝트	20%
출석	10%

프로젝트 = 실습 내용 + 추가 기능

절대 평가

복합 기능, 창의적인 응용 → A0, A+

원리를 모두 이해하고, 각 단원의
모든 기능을 구현 → A-

이해 부족, 일부 구현 미완성
→ B+, B0, B-

과제 미제출, 무성의, 부정행위 등
→ C, D, F

학기 초 대비 학기 말의 성장을 중시

부정 행위는 F 학점 처리

부정 행위의 예시

- 과거 또는 현재 수강 학생으로 부터 코드를 전달 받음.
- AI를 사용하여 생성한 코드를 제출.

서울대 학생

- 이 교과목은 전공선택으로 인정 받지 못함.

오늘의 과제

1. 자기 소개서

- 소속학교/전공/학년
- 관련 기 수강 과목 (논리회로, 컴퓨터구조, 운영체제, 디지털 회로, 회로 설계, 프로그래밍 언어 등)
- 프로그래밍 경험 (C, Python 등)
- Arduino 등 마이크로 컨트롤러 (MCU) 수강 또는 사용 경험
- 수강 목적 (관심 분야, 배우고자 하는 내용, 진로 등)
- 9월 6일 (일) 오후 11:59 마감
- 성적에는 반영 안 됨.

2. 보강 가능 날짜 & 시간

- 다음 페이지의 파란 색 표시 날짜에 3시간 연속 수강이 가능한 시간 표시.

보강 후보 날짜

오늘 < 2026.09 > ☐ 음력 ☐ 손꼽는날 ☐ 기념일

일	월	화	수	목	금	토
30	31	1	2	3	4	5
6	7	8	9	10	11 음 8.1	12
13	14	15	16	17	18	19
20	21	22	23	24	25 음 8.15 추석	26
27	28	29	30	1	2	3

오늘 < 2026.10 > ☐ 음력 ☐ 손꼽는날 ☐ 기념일

일	월	화	수	목	금	토
27	28	29	30	1	2	3 개천절
4	5 대제 휴일	6	7	8	9 한글날	10
11 음 9.1	12	13	14	15	16	17
18	19	20	21	22	23	24
25 음 9.15	26	27	28	29	30	31

오늘 < 2026.11 > ☐ 음력 ☐ 손꼽는날 ☐ 기념일

일	월	화	수	목	금	토
1	2	3	4	5	6	7
8	9 음 10.1	10	11	12	13	14
15	16	17	18	19	20	21
22	23 음 10.15	24	25	26	27	28
29	30	1	2	3	4	5

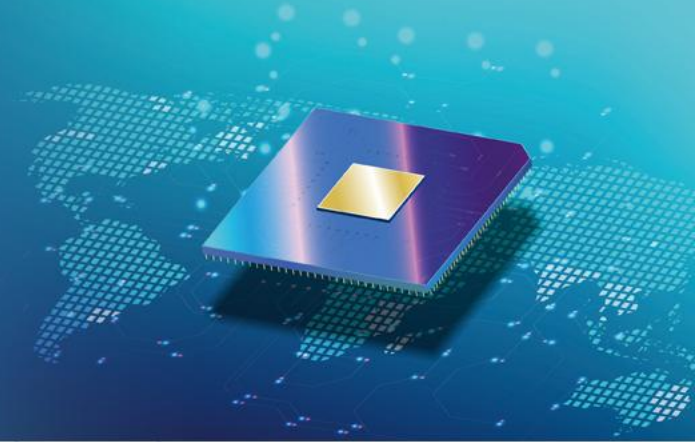
오늘 < 2026.12 > ☐ 음력 ☐ 손꼽는날 ☐ 기념일

일	월	화	수	목	금	토
29	30	1	2	3	4	5
6	7	8	9 음 11.1	10	11	12
13	14	15	16	17	18	19
20	21	22	23 음 11.15	24	25 성탄절	26
27	28	29	30	31	1	2



CHAPTER 2

AVR Microcontroller

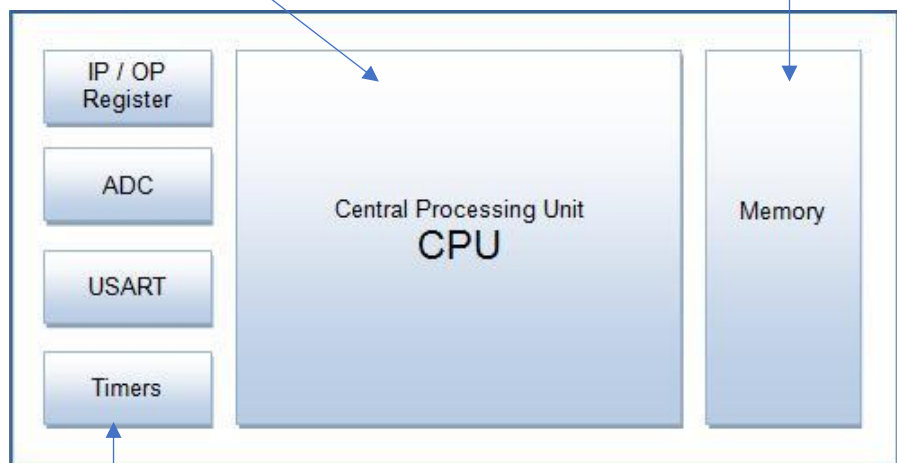


마이크로 컨트롤러 (MCU)

Microcontroller = CPU + 메모리 + 주변장치

프로그램을 저장하는 Flash Memory와, 연산에 필요한 SRAM

작은 크기와 저전력의 중앙 처리 장치 (CPU)



주변장치와 연결할 수 있는 다양한 회로

Leading MCU suppliers (USD million)

2021 Rank	Company	Headquarters	2020	2021	21/20 change (%)	2021 marketshare
1	NXP	Europe	2,980	3,795	27%	18.8%
2	Microchip	USA	2,872	3,584	25%	17.8%
3	Renesas	Japan	2,748	3,420	24%	17.0%
4	ST	Europe	2,506	3,374	35%	16.7%
5	Infineon	Europe	1,953	2,378	22%	11.8%



MICROCHIP

연 매출 \$7.2B (=9조원)
직원 20,000명

\$3.6B (=4.3조원)
에 인수



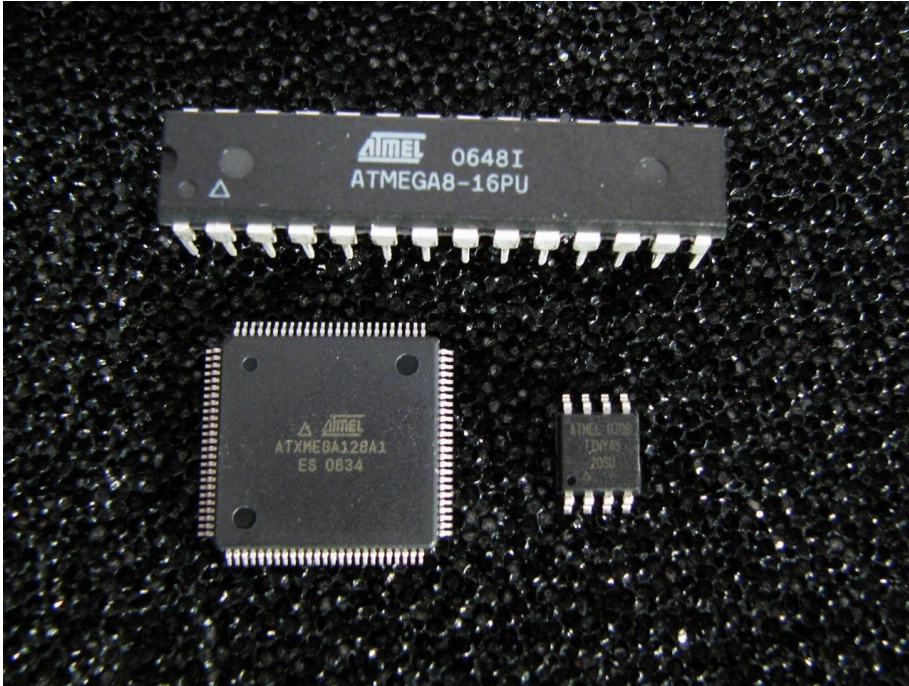
Atmel

연 매출 \$1.2B (=1.4조원)
직원 4,700명



STMicroelectronics
프랑스 + 이탈리아
연 매출 €12.8 (=17조원)
직원 48,000명

AVR Microcontroller



- 노르웨이 공대 (NTH) 학생 2명이 고안 (1997).
- Nordic VSLI 회사가 IC 로 제작.
- Atmel이 기술을 구입.
- Microchip이 Atmel 을 인수.



- 8-bit RISC Architecture
- Flash Memory 를 내장한 최초의 microcontroller.
- 2003년에 이미 5억 개 판매.
- Arduino 보드에 사용 (ATmega).

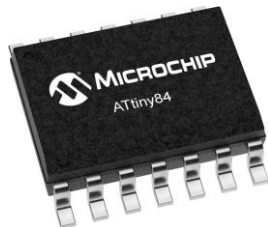
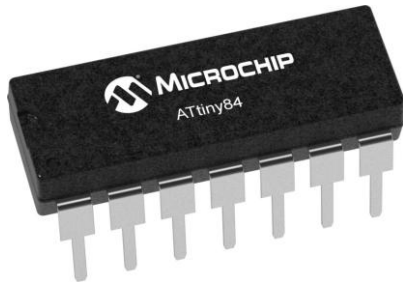
ATtiny Series

- AVR의 일종으로, 낮은 사양의 집적회로 (IC) 반도체.
- 가장 기본적인 주변장치 (Peripherals) 기능만 탑재.
 - GPIO, USI, ADC, Timer, Interrupt Controller
- 최소한의 Flash Memory 와 SRAM 탑재.

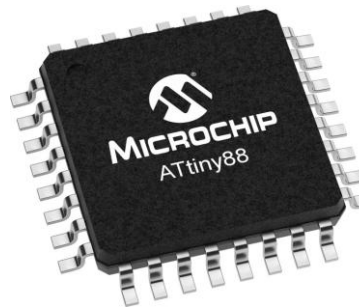
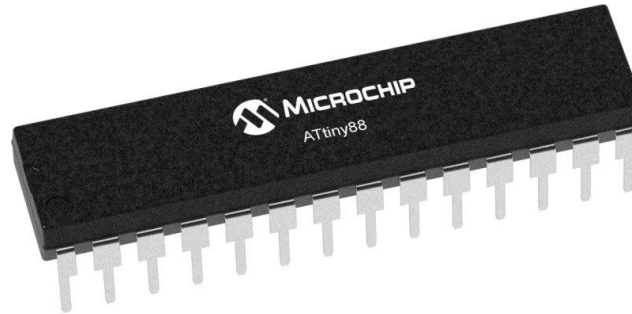
ATtiny85



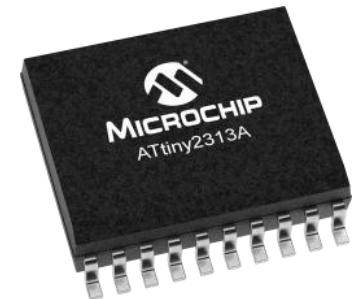
ATtiny84



ATtiny88

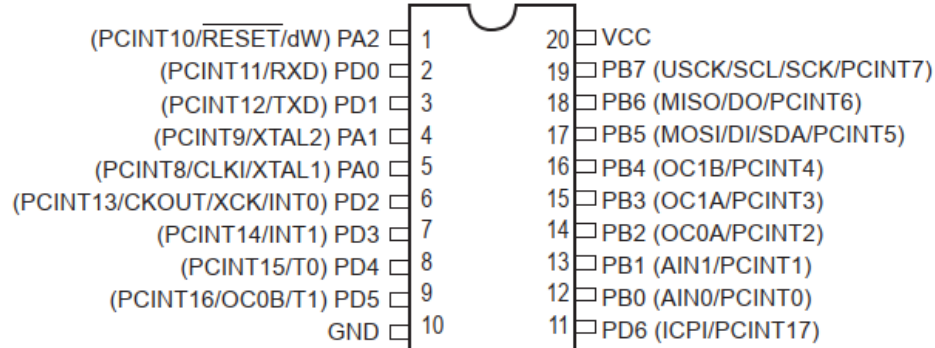


ATtiny2313A



ATtiny4313-PU

Pinout



Major Features

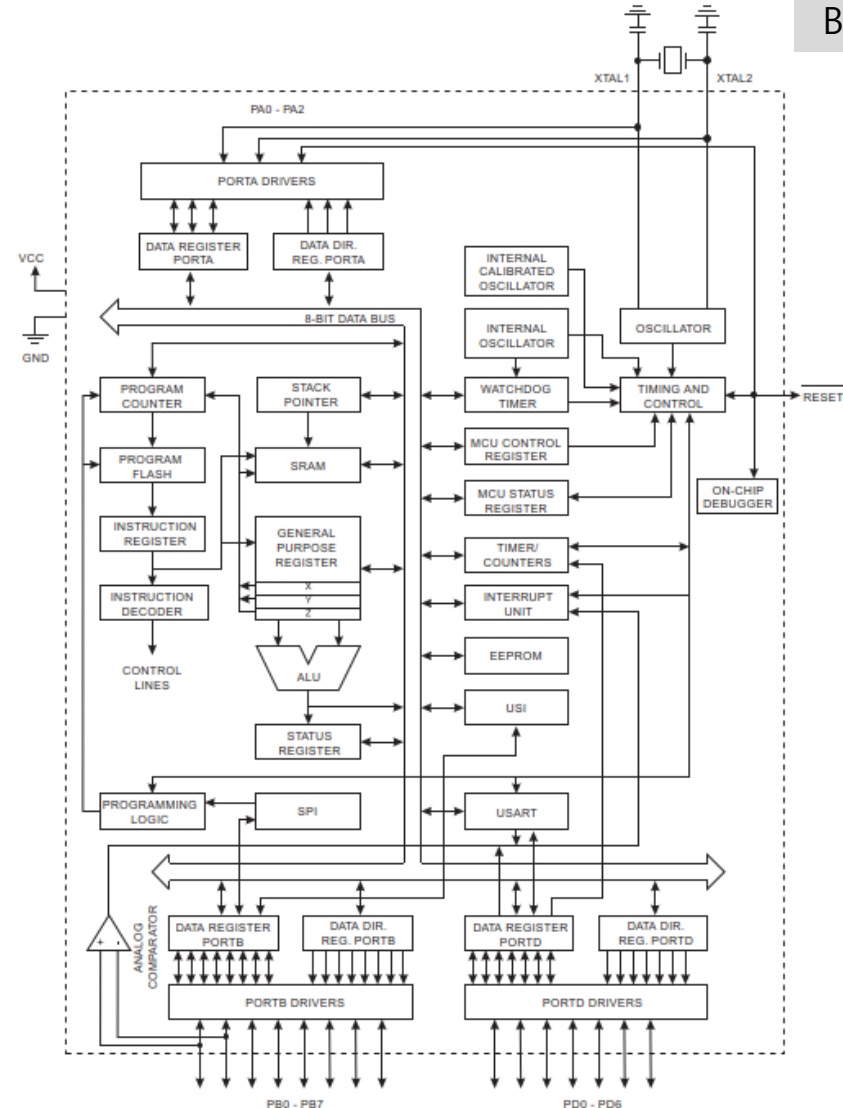
- AVR CPU Core (120 instructions)
- 4k bytes of Program Memory (Flash)
- 256 bytes of SRAM
- Two timers
- USART
- Universal Serial Interface (USI)
- 18 Pin-Change Interrupts
- In-System Programming
- Internal Calibrated Oscillator
- 2.7V~5.5V Operating Voltage
- 0-20MHz Operating Frequency

우리 회로

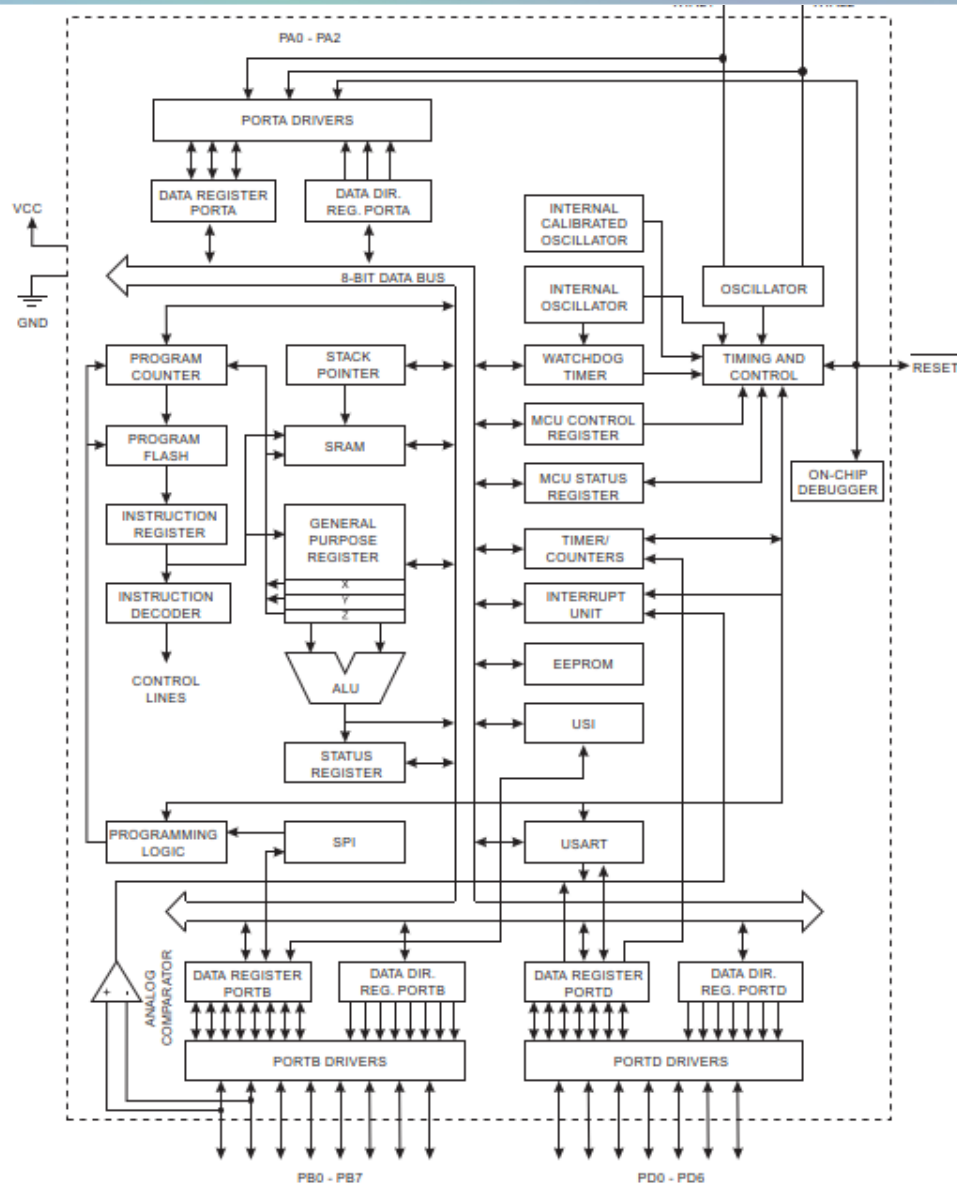
- 5V로 동작.
- 내부 Clock을 사용하여 8MHz 로 동작.

Block Diagram

Datasheet
Page 5



ATtiny4313로 수업할 내부 하드웨어



GPIO (General Purpose Input Output)

USI (Universal Serial Interface)

USART (Universal Sync/Async Receiver & Transmitter)

Interrupt

Timers

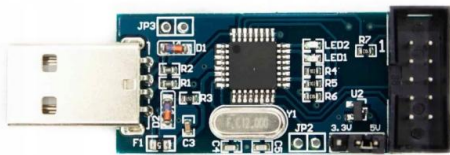
ATtiny 개발 하드웨어와 소프트웨어

하드웨어

ATtiny IC

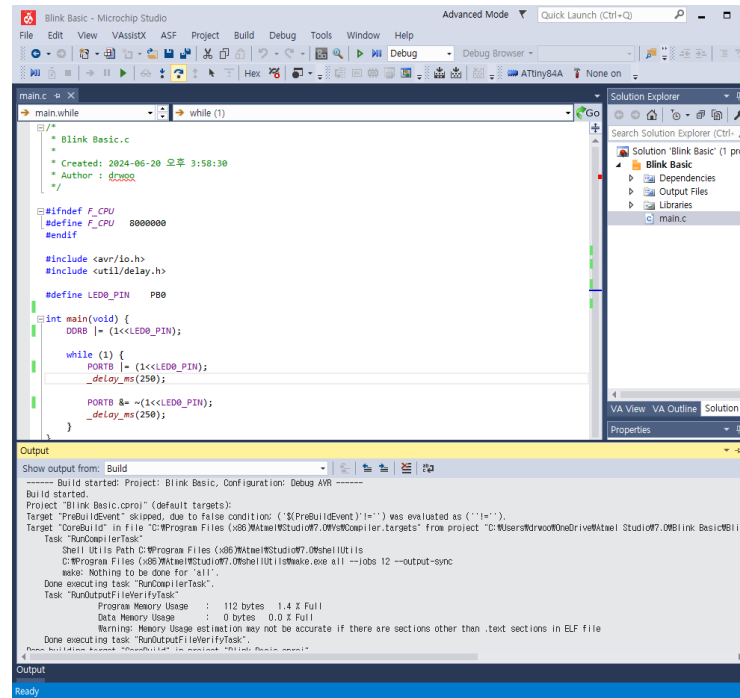


프로그래머 (USBasp)



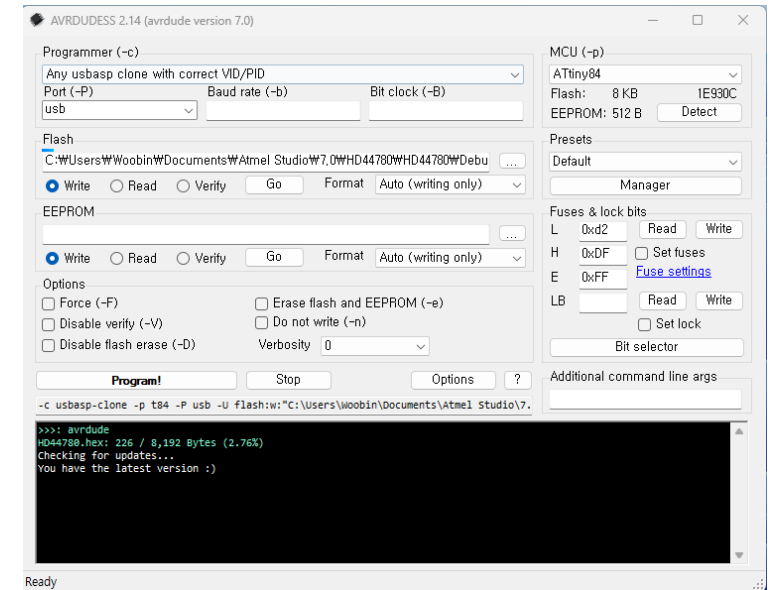
소프트웨어

Microchip Studio



소스 코드를 C/C++ 언어로 작성하고,
그 소스 코드를 기계어 코드로 변환하
는 통합 소프트웨어.

AVRDUDESS



기계어 코드 파일을 Attiny IC에
업로드하는 소프트웨어.

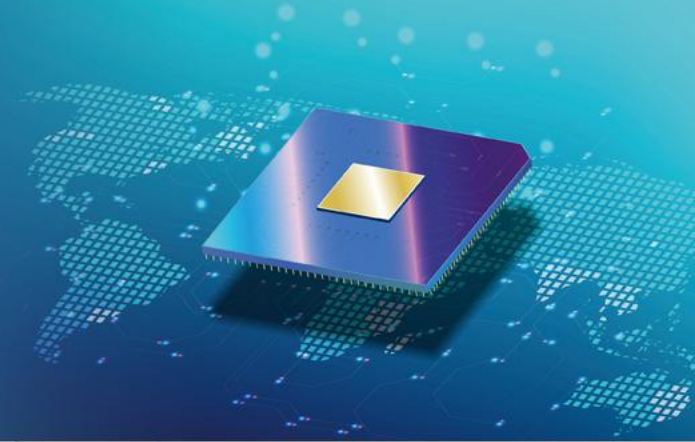
ATtiny 프로그래밍 흐름





CHAPTER 3

ATtiny 기본 동작



ATtiny 하드웨어와 소프트웨어 구현 순서

준비 과정 (1번만 수행)

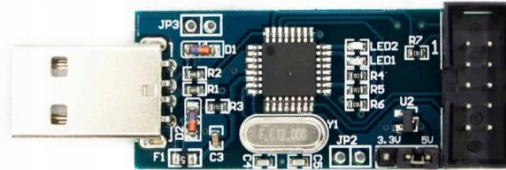
1. Microchip Studio와 AVRDUDESS 설치.
2. ~~USBasp 드라이버 설치.~~
3. ATtiny IC 와 USBasp의 핀들을 연결.
4. AVRDUDESS를 실행하여 USBasp를 통해 ATtiny IC와 통신이 되는지 확인.
5. ATtiny IC 내부의 Fuse Byte를 재설정하여 ATtiny Clock 속도를 높임.

반복 과정

1. 회로를 구성하여 ATtiny IC와 부품들을 연결.
2. Microchip Studio에서 소스 코드 작성.
3. Microchip Studio에서 소스 코드를 Build 하여 기계어 코드 (Hex file)을 생성.
4. AVRDUDESS를 실행하여 기계어 코드를 ATtiny IC에 업로드.
5. 업로드가 끝나면 ATtiny IC는 리셋되고 코드가 자동 실행됨.

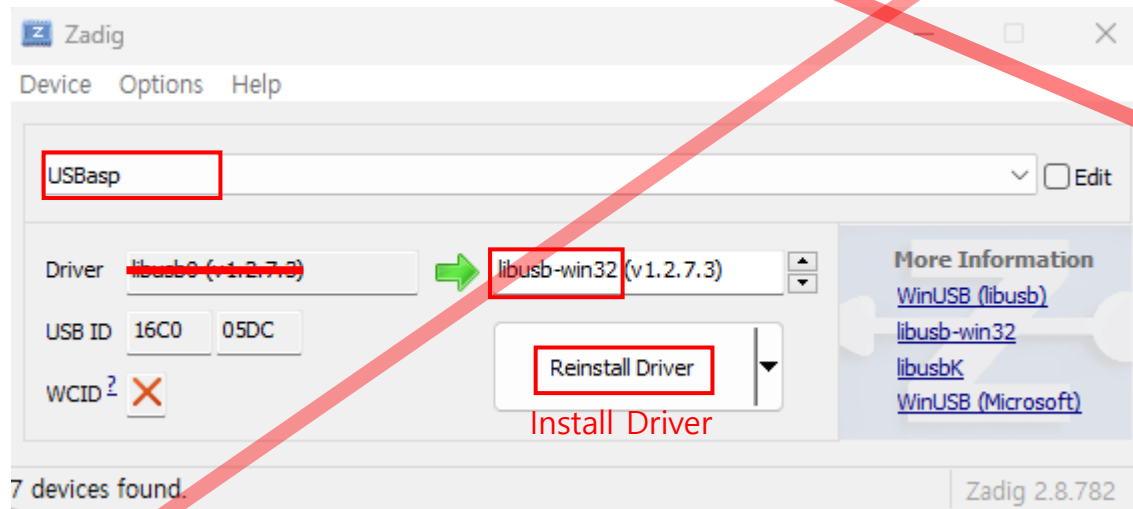
USBasp 드라이버 설치

① USBasp를 USB 포트에 삽입

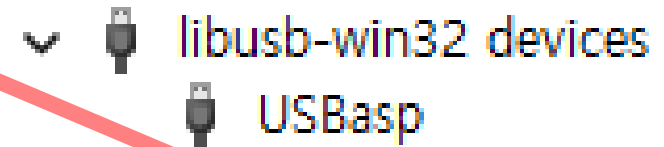


USBasp firmware를 업데이트 하였기에 드라이버 설치가 필요하지 않음.

② Zadig를 실행하여 드라이버 설치



③ 윈도우 장치관리자에서 USBasp 드라이버가 설치됨을 확인.



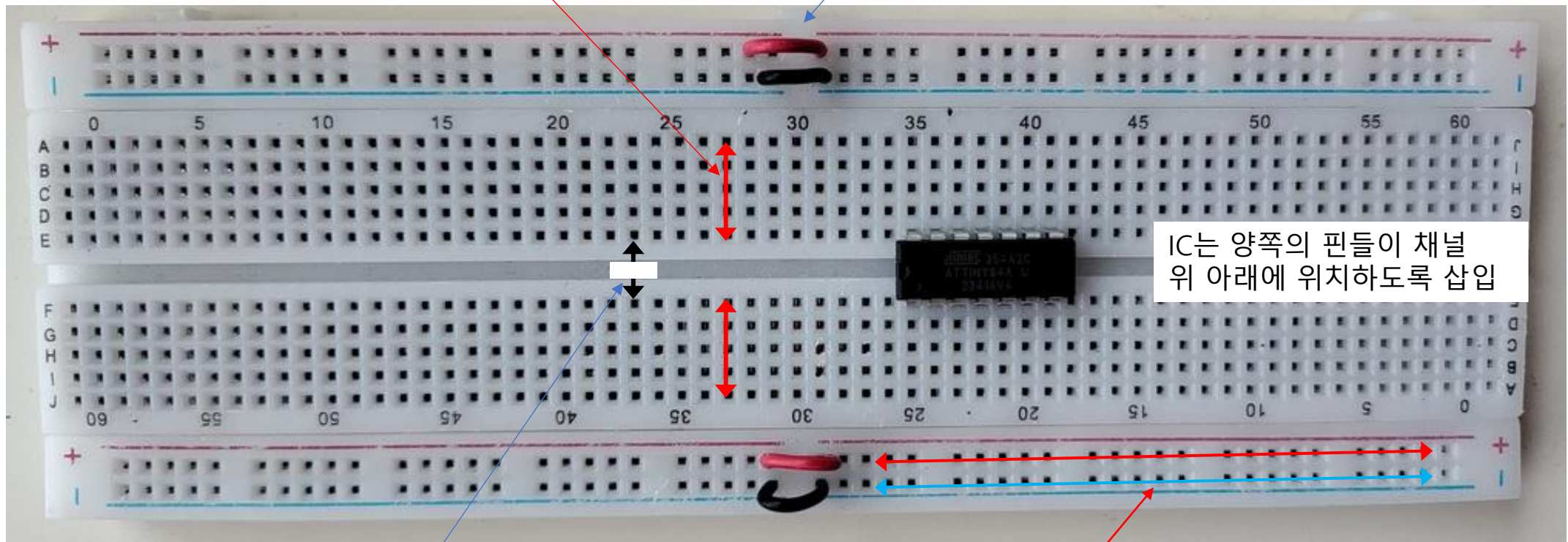
④ 설치 확인 후 USBasp 를 USB 포트에서 분리.

위 예제는 이미 설치된 PC 에서 실행하였기에 Reinstall 로 나오지만, 처음 실행시에는 Install Driver 가 표시됨.

Breadboard (빵판)

홀들은 세로 방향으로 전기적으로 연결되어 있음.

가로 방향의 홀들은 판 중간에서 좌/우가 분리되어 있으므로, 케이블로 연결 필요.

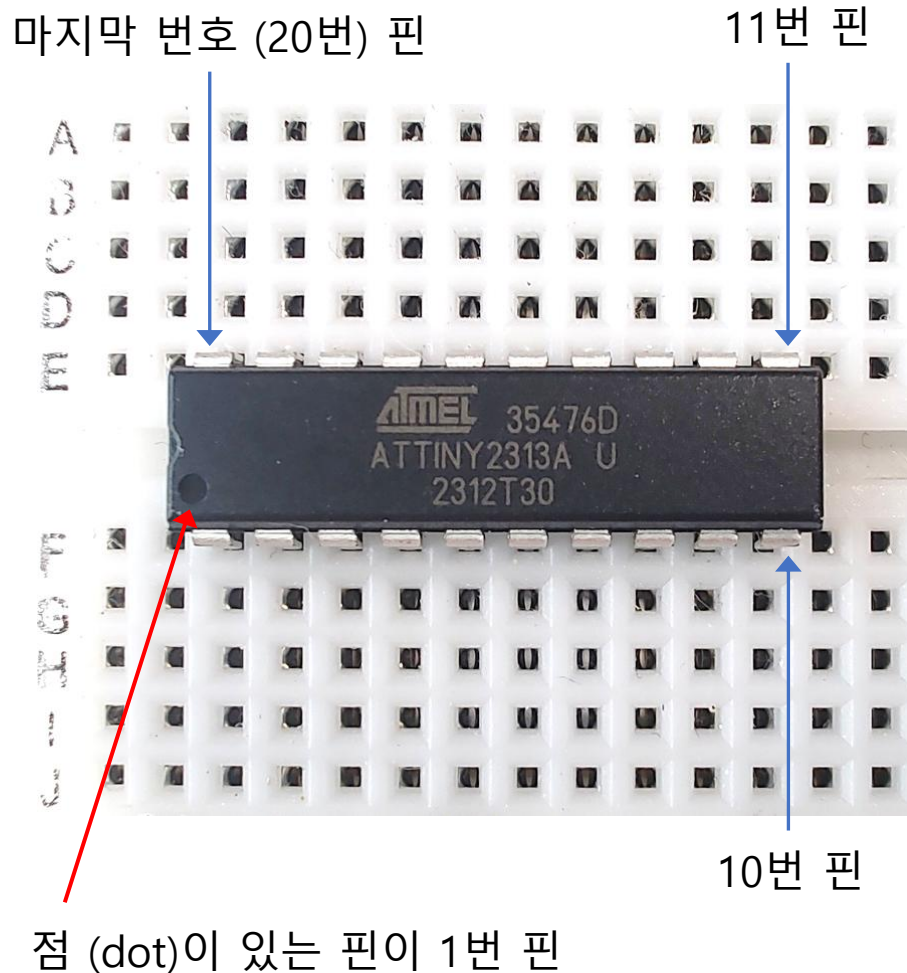


중앙 채널의 위 아래는 전기적으로 분리되어 있음.

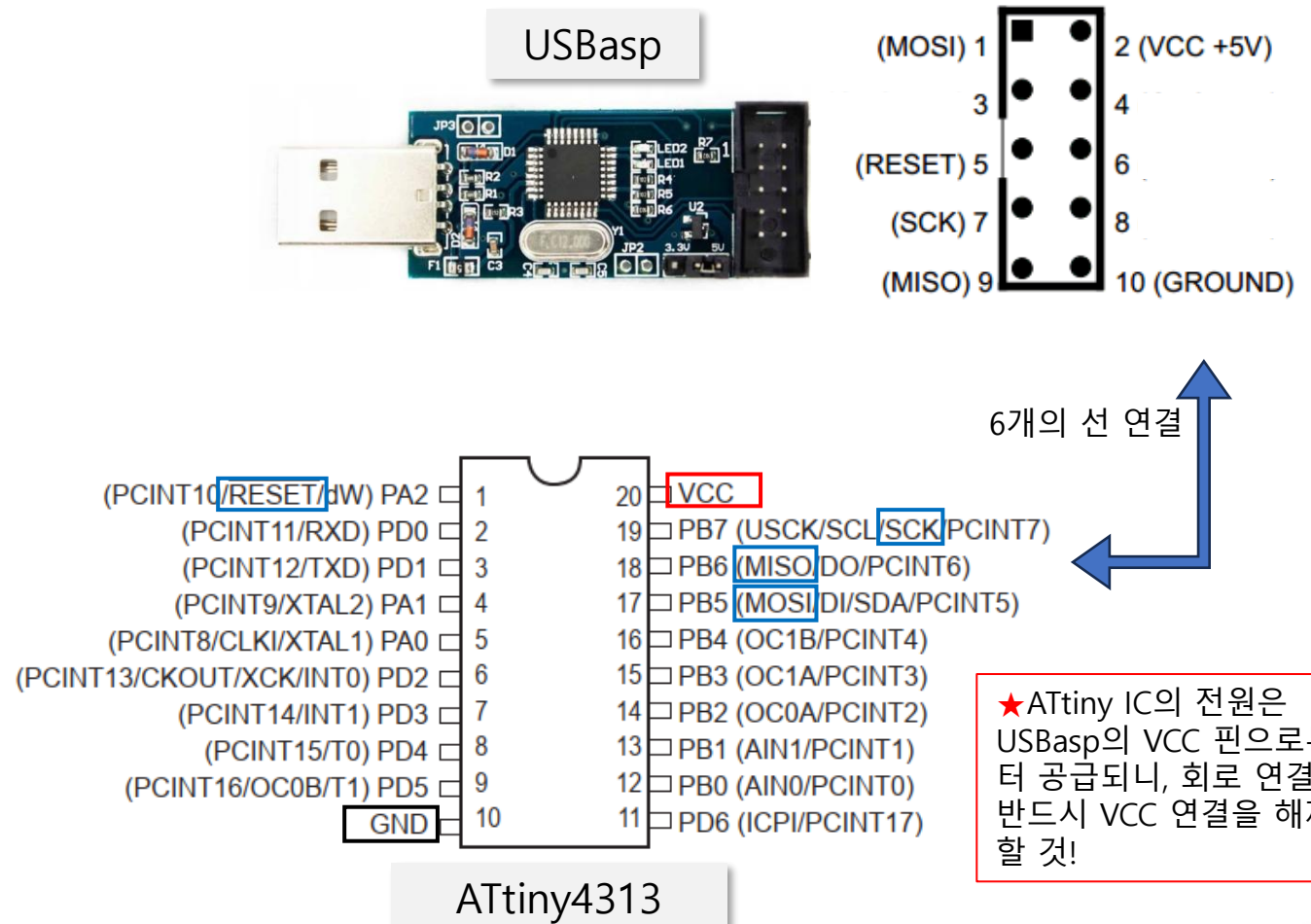
위 아래 마지막 2줄의 홀들은 가로 방향으로 연결되어 있음 (VCC 및 Ground로 사용).

ATtiny와 USBasp 프로그래머 연결

① ATtiny IC를 Breadboard에 삽입



② USBasp의 커넥터와 ATtiny IC를 연결



ATtiny4313 Clock 설정

Fuse Bytes:

Clock Source, Reset 등 ATtiny의 초기 기능 설정을 저장하는 메모리 영역.

- ATtiny IC는 공장 출고 시 내부 RC Oscillator가 8MHz로 발진하고, 그 출력을 8로 나눈 1MHz로 동작하도록 초기 설정되어 있음.
- 내부 RC Oscillator 선택은 유지하되, CKDIV8 bit를 1로 재설정하여 8MHz로 동작하게 변경할 것임.

Clock Source 선택

Clock Option	Frequency
내부 128kHz RC Oscillator	128kHz
내부 RC Oscillator	4MHz / 8MHz
외부 Crystal	0~20MHz
외부 Oscillator	0~20MHz

Table 20-4. High Fuse Byte

Bit #	Bit Name	Use	See	Default Value
7	DWEN	Enables debugWIRE ⁽¹⁾	Page 170	1 (unprogrammed)
6	EESAVE	Preserves EEPROM memory during Chip Erase operation	Page 187	1 (unprogrammed) ⁽²⁾
5	SPIEN	Enables serial programming and downloading of data to device ⁽³⁾		0 (programmed) ⁽⁴⁾
4	WDTON	Sets watchdog timer permanently on	Page 45	1 (unprogrammed)
3	BODLEVEL2	Sets BOD trigger level	Page 202	1 (unprogrammed)
2	BODLEVEL1			1 (unprogrammed)
1	BODLEVEL0			1 (unprogrammed)
0	RSTDISBL	Disables external reset ⁽¹⁾	Page 41	1 (unprogrammed)

Table 20-5. Low Fuse Byte

Bit #	Bit Name	Use	See	Default Value
7	CKDIV8	Divides clock by 8 ⁽¹⁾	Page 28	0 (programmed)
6	CKOUT	Outputs system clock on port pin	Page 32	1 (unprogrammed)
5	SUT1	Sets system start-up time	Pages 28, 29, and 31	1 (unprogrammed)
4	SUT0			0 (programmed) ⁽²⁾
3	CKSEL3	Selects clock source	Page 27	0 (programmed) ⁽³⁾
2	CKSEL2			0 (programmed) ⁽³⁾
1	CKSEL1			1 (unprogrammed) ⁽³⁾
0	CKSEL0			0 (programmed) ⁽³⁾

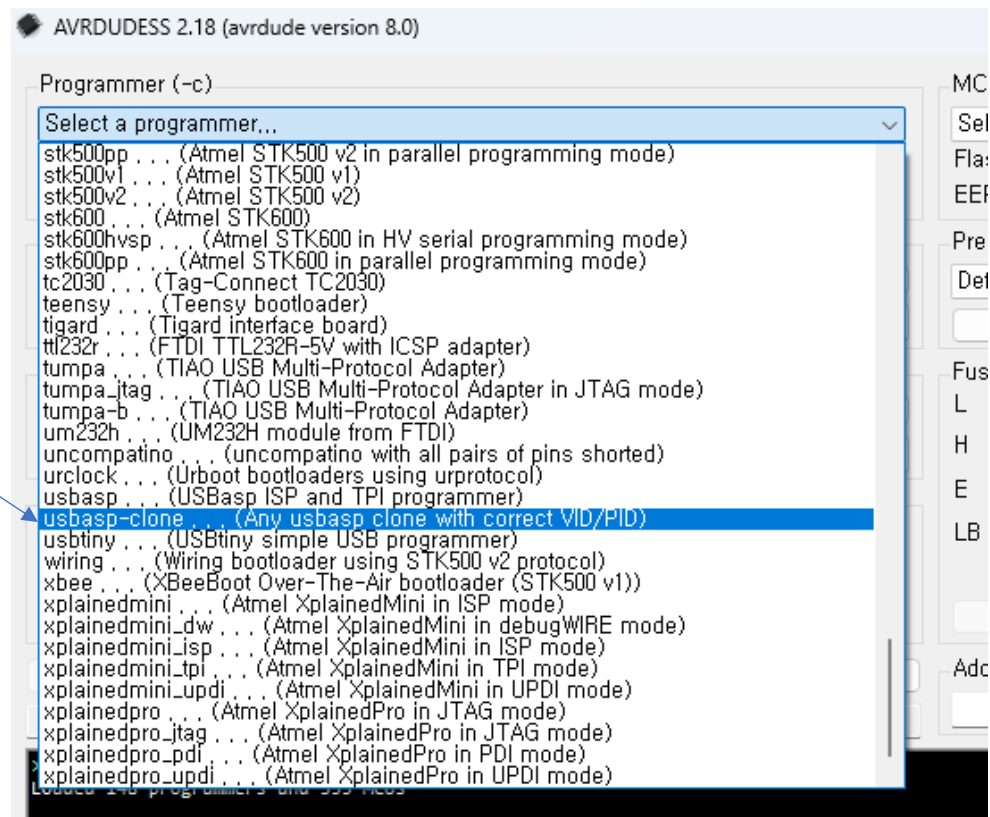
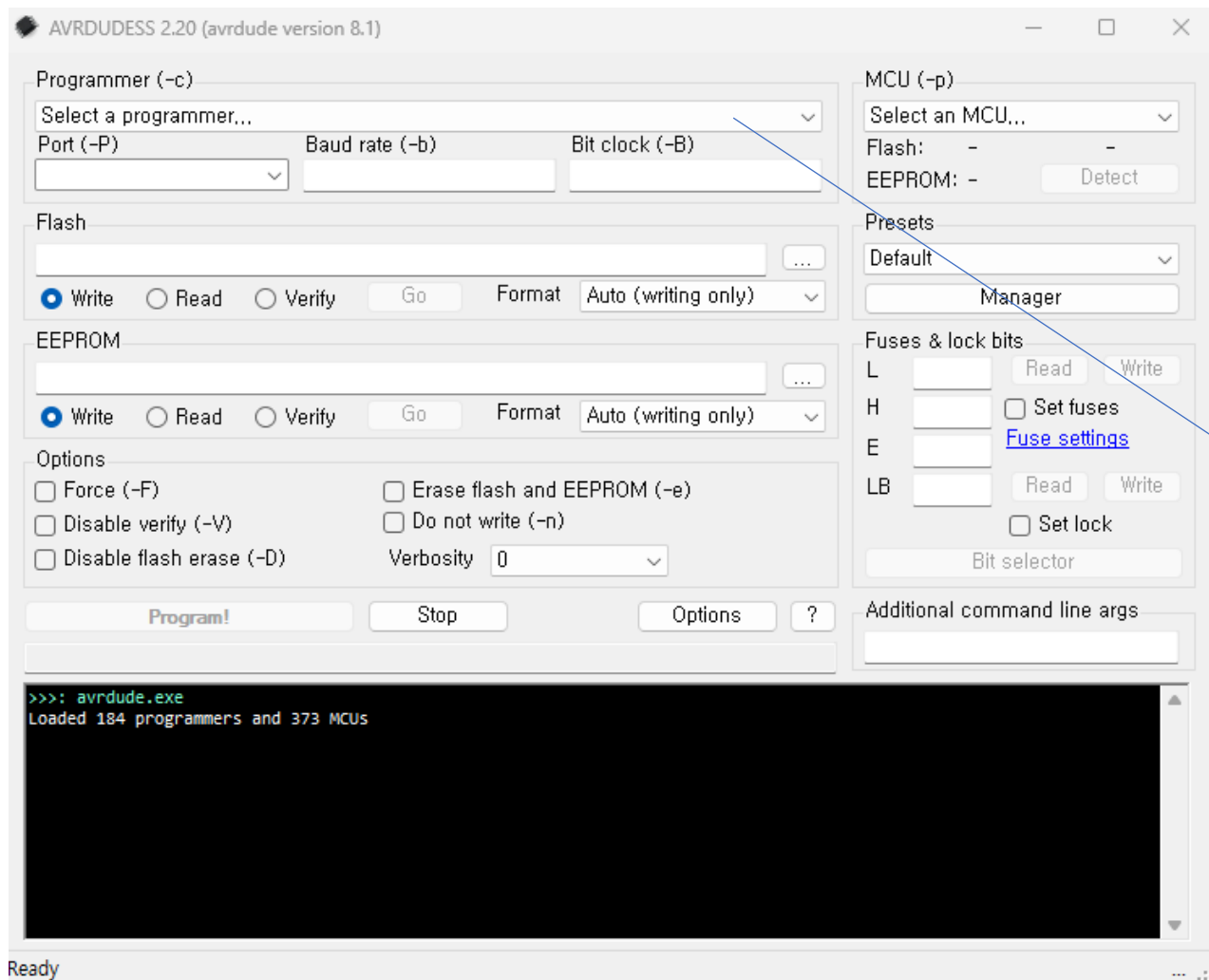
→ 1로 재설정

AVRDUDESS 실행

① USBasp를 USB 포트에 삽입

② AVRDUDESS 실행

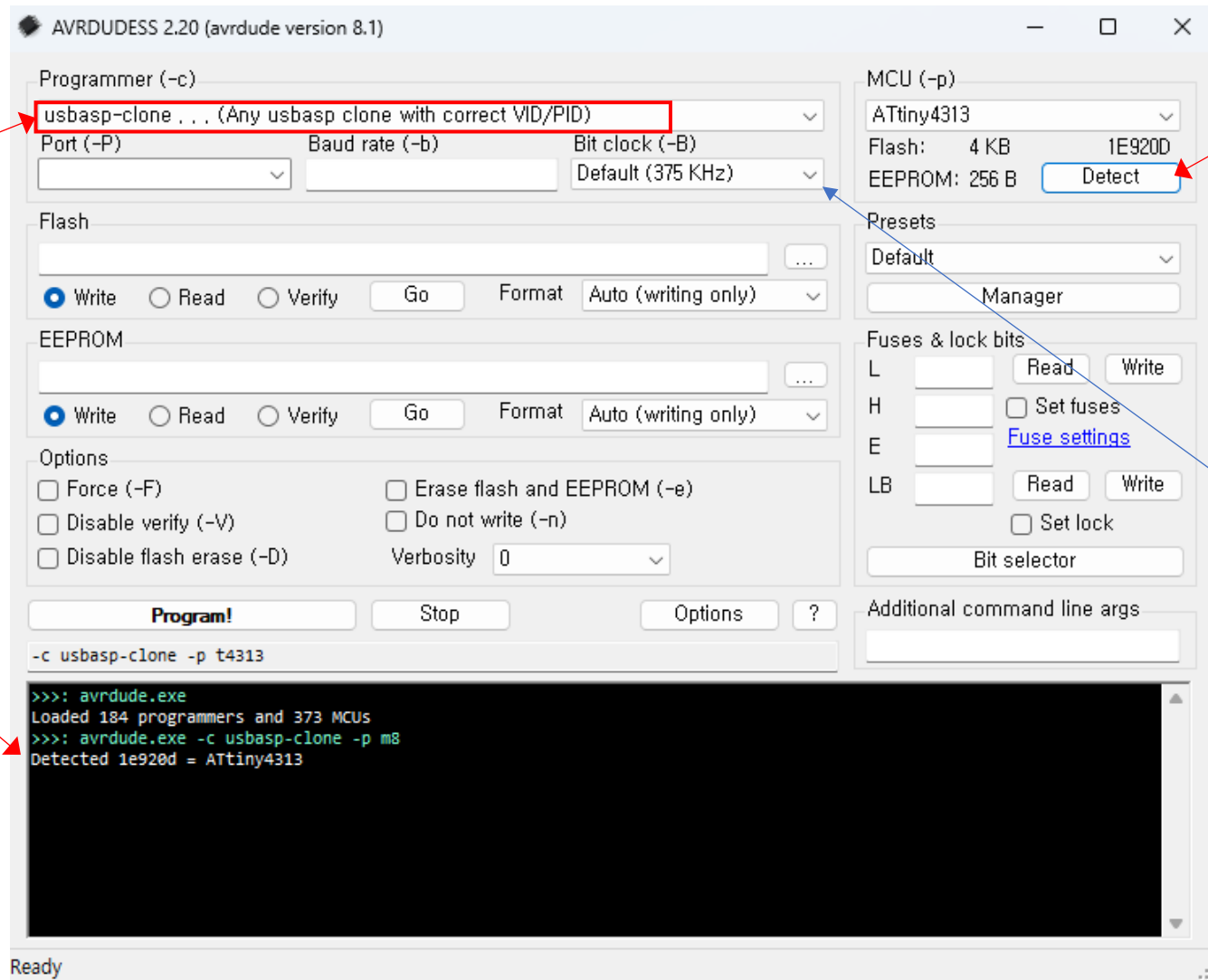
③ USBASP 선택



ATtiny IC 와 통신 확인

① Programmer
기종 선택

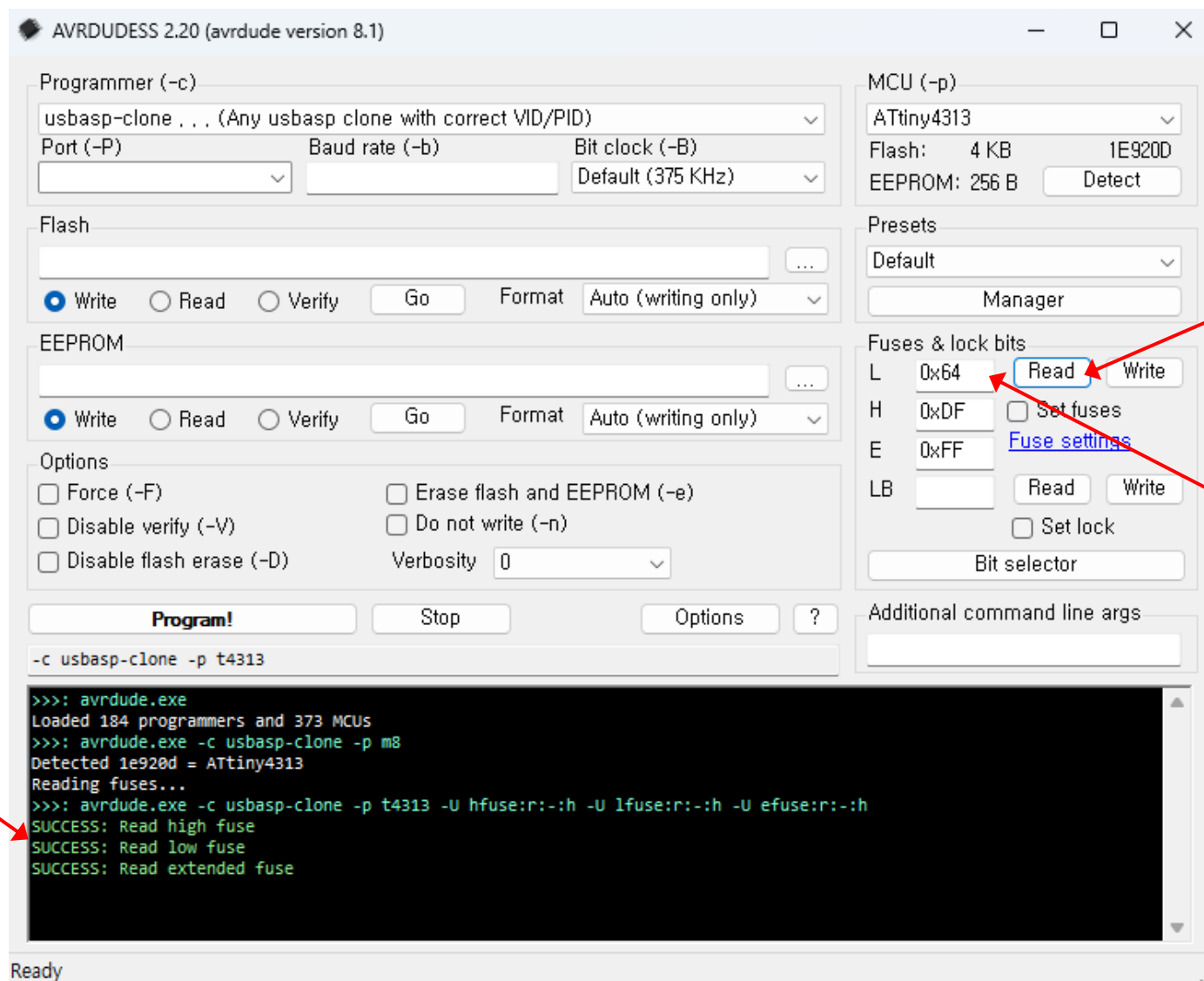
③ ATtiny4313이
인식됨을 확인



② Detect 버튼 누름

★ ATtiny4313이 인식
되지 않는 경우, Bit
clock을 32kHz로 낮춰
서 시도.

Fuse Bits의 Read



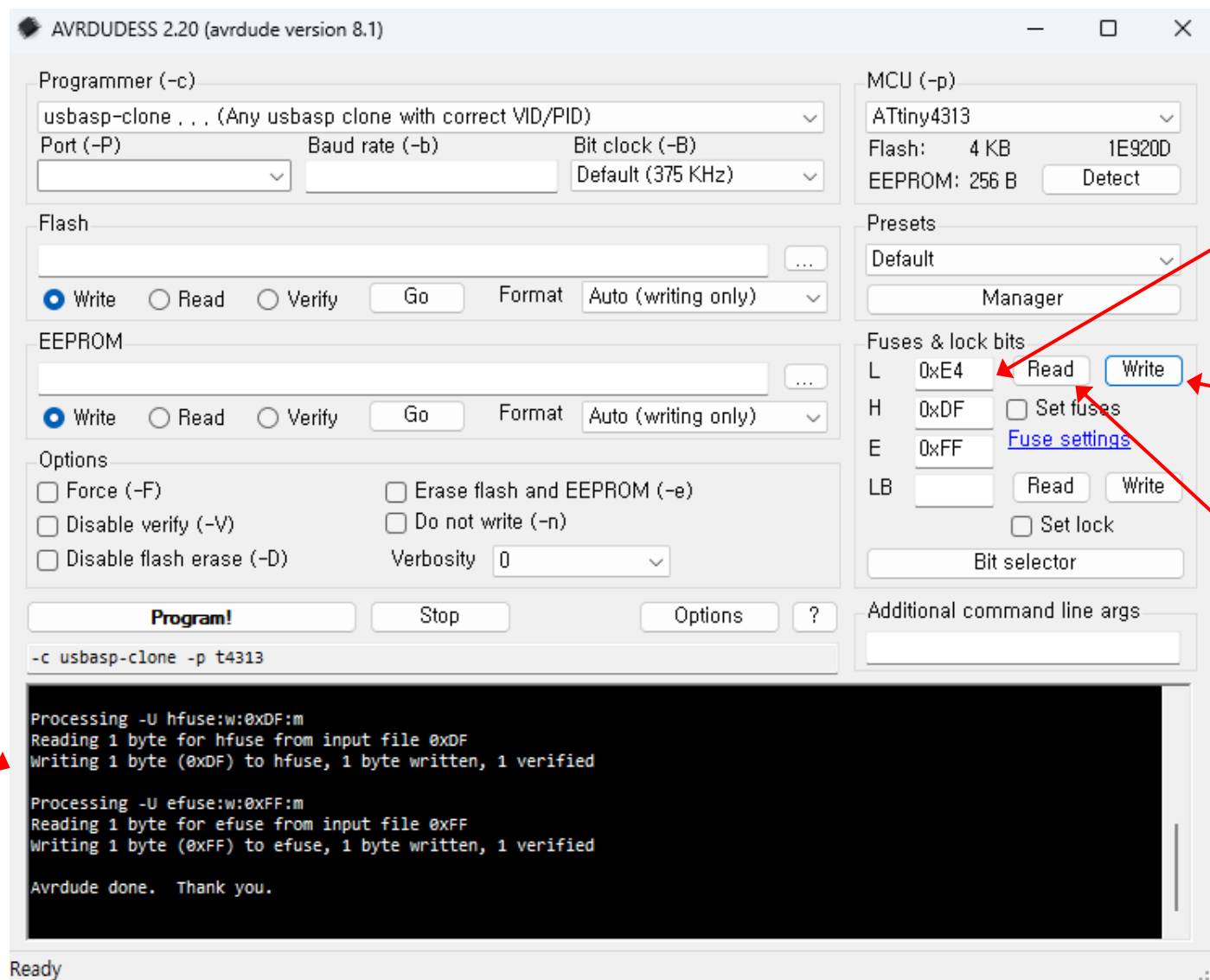
② Fuse Byte의 Read
가 실행됨을 확인.

① Fuse Bits의 Read
버튼을 누름

③ Fuse Bits 의 Low
Byte (L) 가 0x64로
Read 됨을 확인.

0x64 = 1MHz 동작

8MHz 동작을 위한 Fuse Bits의 설정



③ Fuse Bits가 write 됨을 확인.

① Fuse Bits의 Low Byte (L) 에 0xE4를 입력

② Write 버튼을 누름

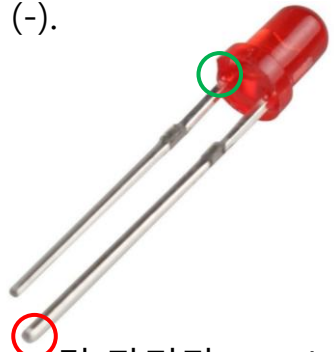
④ Read 버튼을 누르면 Low Byte (L) 이 0xE4로 유지됨을 확인

★ Fuse Bits 설정은 새 IC 에 한번만 하면 됨.

첫 실습: LED Blink

LED

잘린 단면이 cathode (-).



긴 다리가 anode (+).

Anode (+)  Cathode (-)

저항 (resistor)

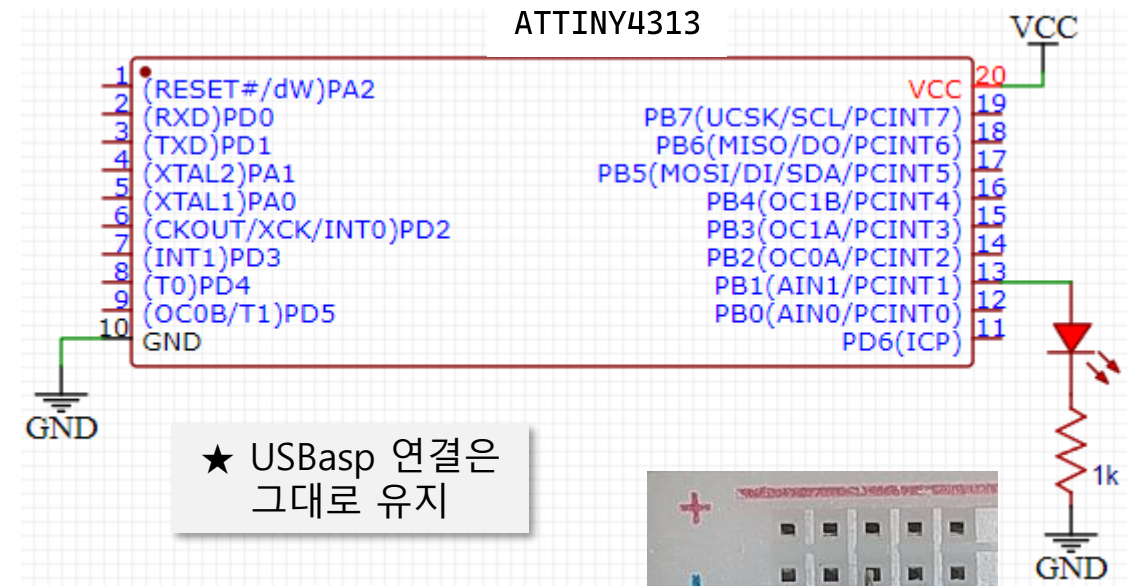
1K ohm 저항의 표기는 Brown, Black, Red 띠.



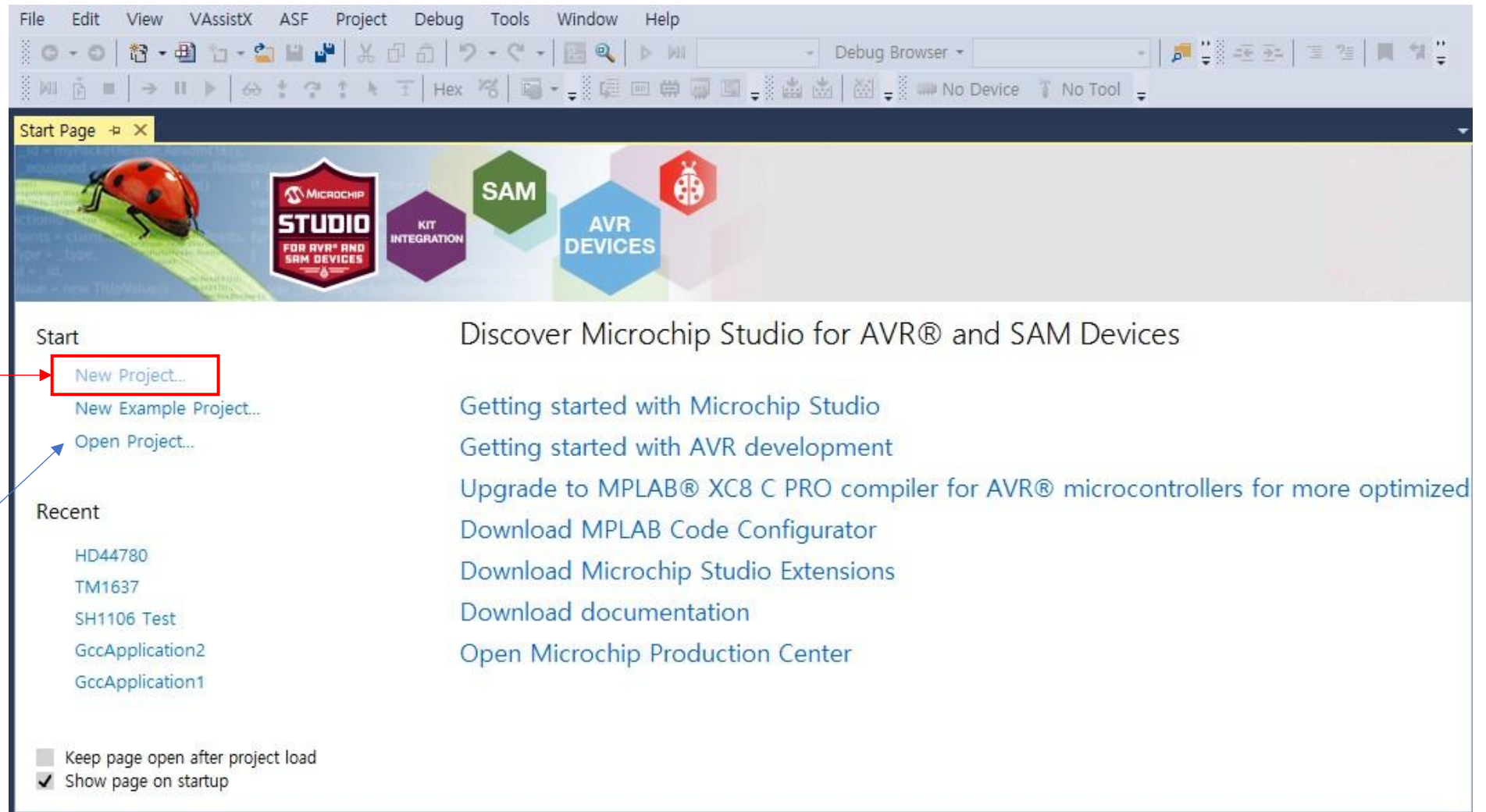
극성은 없음.



회로 연결



Microchip Studio 실행



New Project
클릭

이미 작성한 코드를
수정할 때는 Open
Project 클릭

신규 Project 생성

New Project

Recent

Installed

C/C++

Assembler

Microchip Studio Solution

Sort by: Default

AVR XC8 C Application Project

AVR XC8 C Library Project

Convert AVR GCC to XC8 Project

GCC C ASF Board Project

GCC C Executable Project

GCC C Static Library Project

GCC C++ Executable Project

GCC C++ Static Library Project

Create project from Arduino sketch

Device Selection

Device Family: All

ATTINY4

Name	App./Boot Memory (Kbytes)	Data Memory (bytes)	EEPROM (bytes)
ATtiny406	4	256	128
ATtiny412	4	256	128
ATtiny414	4	256	128
ATtiny416	4	256	128
ATtiny416auto	4	256	128
ATtiny417	4	256	128
ATtiny424	4	512	128
ATtiny426	4	512	128
ATtiny427	4	512	128
ATtiny4313	4	256	256
ATtiny43U	4	256	64
ATtiny44	4	256	256
ATtiny441	4	256	256
ATtiny44A	4	256	256
ATtiny45	4	256	256
ATtiny461	4	256	256
ATtiny461A	4	256	256
ATtiny48	4	256	64

Device Info:

Device Name: ATtiny4313

Speed: N/A

Vcc: N/A

Family: ATtiny

[Device page for ATtiny4313](#)

[Datasheet](#)

Supported Tools

- Atmel-ICE
- AVR Dragon
- AVR ONE!
- EDBG
- EDBG MSD
- AVRISP mkII
- JTAGICE3

OK Cancel

이름 수정

Name: **Blink**

Location: c:\users\drwoow\Documents\Atmel Studio\7.0

Solution name: Blink

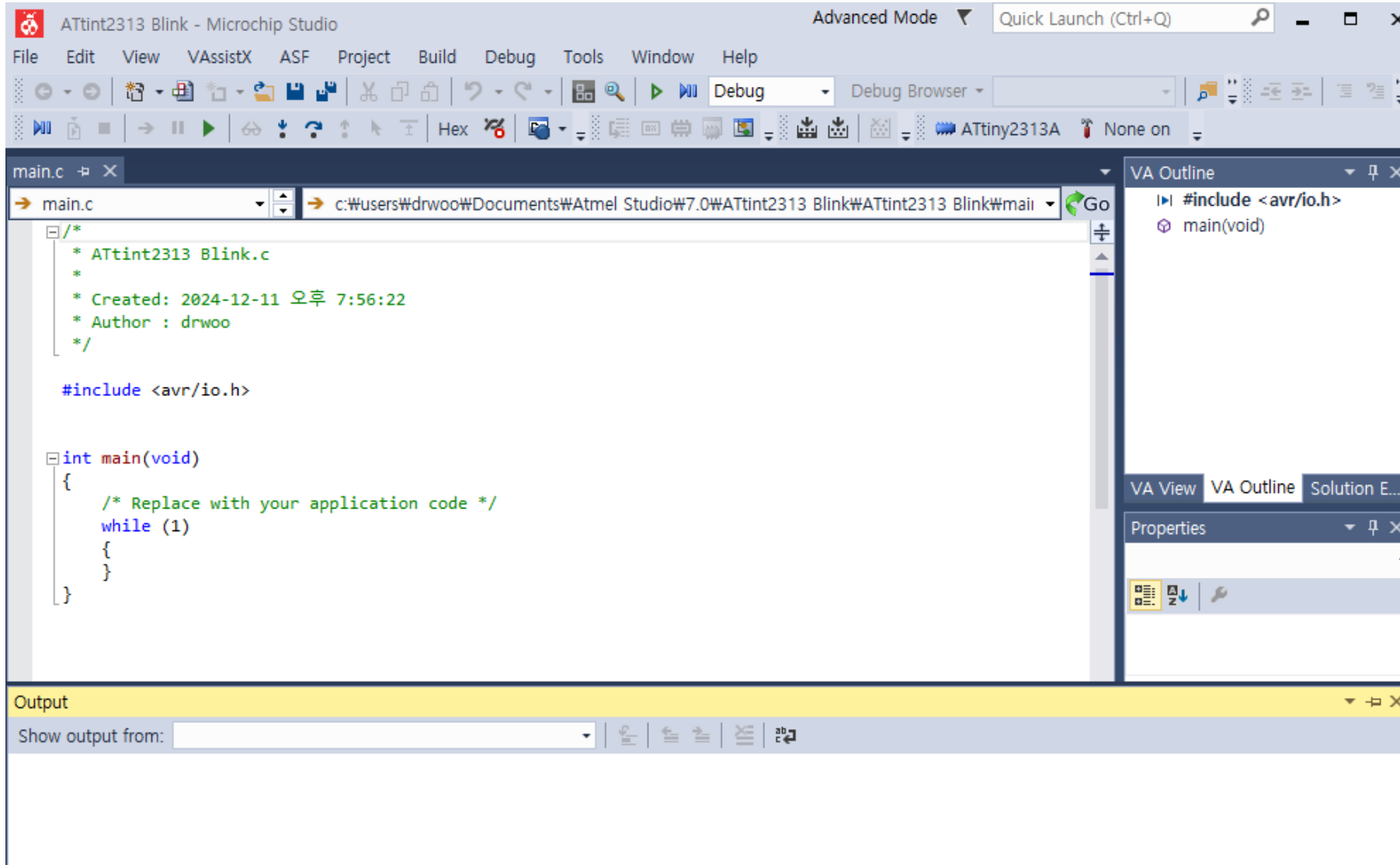
Browse...

☒ Create directory for solution

OK

OK 누르면 부품 선택 창이 열림

Code Editor 화면



LED Blink Code

```
#define F_CPU 8000000 ← Clock=8MHz를 알려줌.

#include <avr/io.h> ← I/O Port 이름을 사용하기 위해 include.
#include <util/delay.h> ← _delay_ms() 함수를 사용하기 위해 include.

int main(void) {
    DDRB |= 0b00000010; // DDRB Bit 1 ← 1

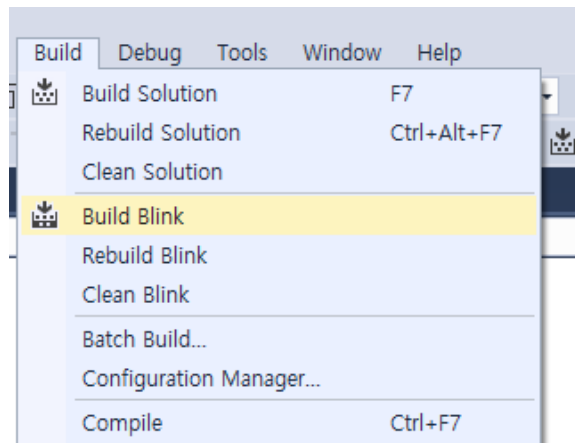
    while (1) {
        PORTB |= 0b00000010; // PORTB Bit 1 ← 1
        _delay_ms(500); // Wait 500ms

        PORTB &= 0b11111101; // PORTB Bit 1 ← 0
        _delay_ms(500); // Wait 500ms
    }
}
```

DDRB	Bit	7	6	5	4	3	2	1	0
	0x17 (0x37)	DDB7	DDB6	DDB5	DDB4	DDB3	DDB2	DDB1	DDB0
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Initial Value	0	0	0	0	0	0	0	0
PORTB	Bit	7	6	5	4	3	2	1	0
	0x18 (0x38)	PORTB7	PORTB6	PORTB5	PORTB4	PORTB3	PORTB2	PORTB1	PORTB0
	Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
	Initial Value	0	0	0	0	0	0	0	0

기계어 파일 생성 (Build)

Build 시작



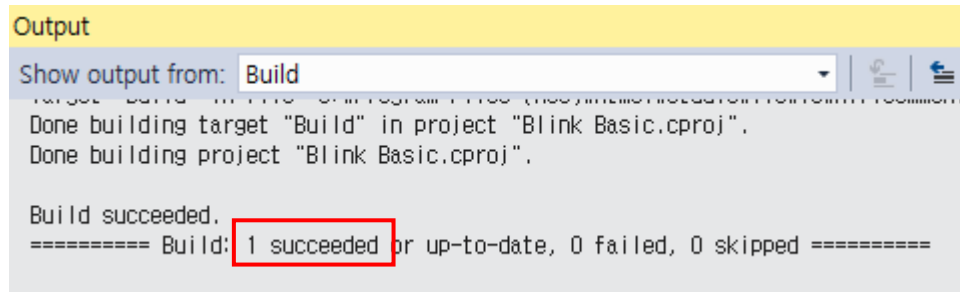
Build Solution, Build Blink 모두 가능



Debug 폴더에 생성되는 파일

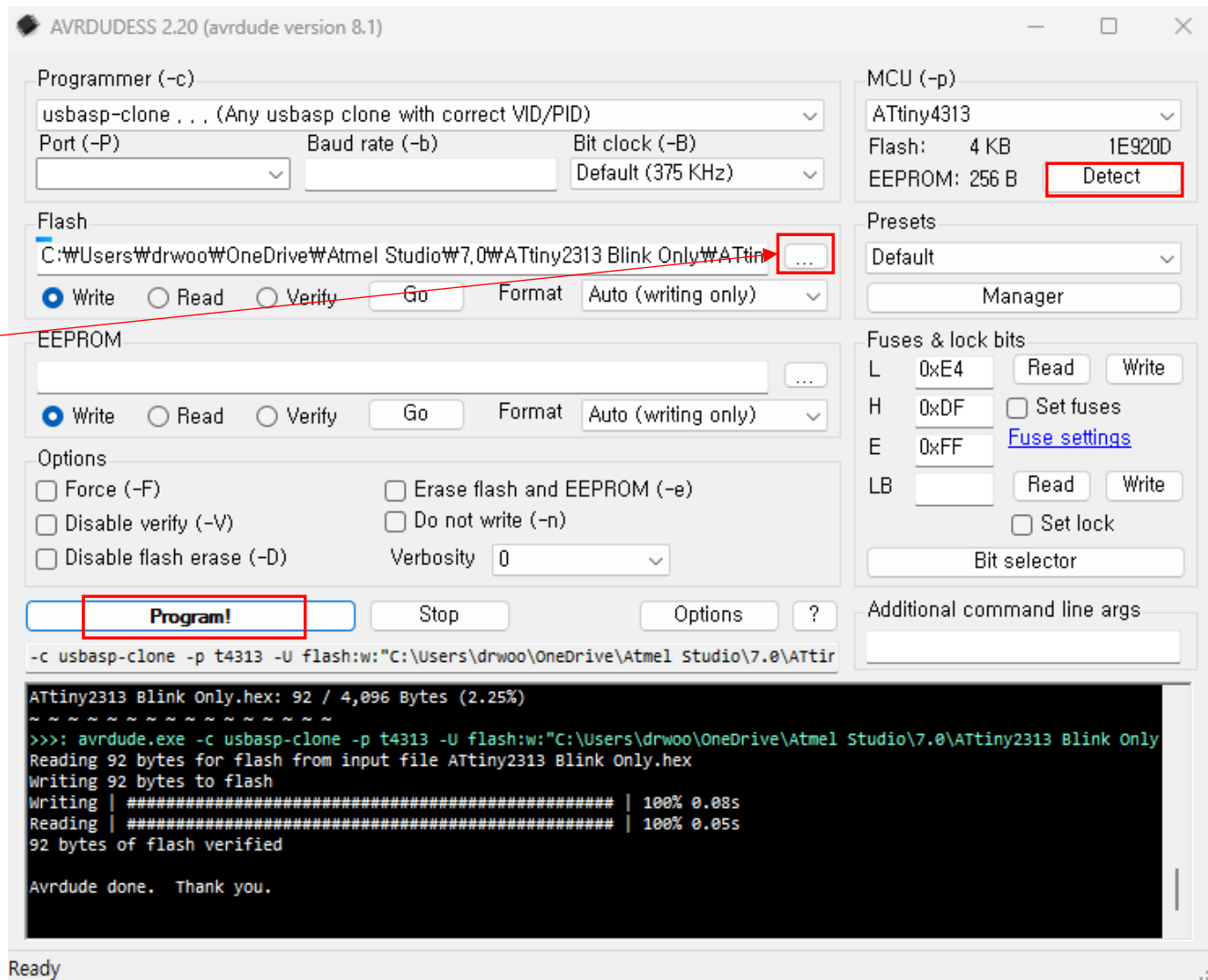
문서 > Atmel Studio > 7.0 > Blink > Blink > Debug			
이름	수정한 날짜	유형	크기
Blink.eep	2023-07-18 오후 3:32	Eep 파일	1KB
Blink.elf	2023-07-18 오후 3:32	ELF 파일	7KB
Blink.hex	2023-07-18 오후 3:32	HEX 파일	1KB
Blink.lss	2023-07-18 오후 3:32	LSS 파일	4KB
Blink.map	2023-07-18 오후 3:32	MAP 파일	14KB
Blink.srec	2023-07-18 오후 3:32	SREC 파일	1KB
main.d	2023-07-18 오후 3:32	D 파일	3KB
main.o	2023-07-18 오후 3:32	O 파일	4KB
makedep	2023-07-18 오후 3:29	Makefile ...	1KB
Makefile	2023-07-18 오후 3:33	파일	4KB

*.hex 파일을
ATtiny에 업로드
할 것임.



Build 성공 확인

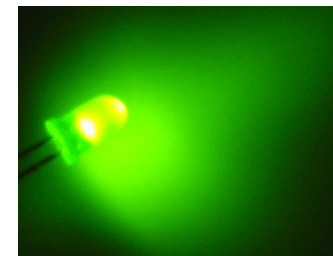
기계어 파일 업로드



② 파일 경로 버튼을 눌러서 Blink.hex를 지정.

③ Program 버튼을 누르면 기계어 코드가 ATtiny로 업로드 됨.

① Detect 를 눌러서 ATtiny4313이 인식됨을 확인.



④ 코드 업로드가 끝나면 Blink 프로그램이 ATtiny에서 자동 실행됨.

LED Blink Code 단순화 (PINB 반전 기능 사용)

```
#define F_CPU 8000000

#include <avr/io.h>
#include <util/delay.h>

int main(void) {
    DDRB |= 0b00000010;    // DDRB Bit 1 ← 1

    while (1) {
        PINB = 0b00000010; // PINB Bit 1 ← 1
        _delay_ms(500);     // Wait 500ms
    }
}
```

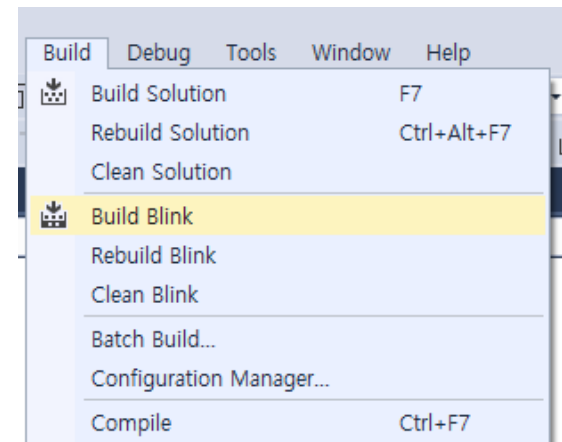
★ PINB에 Write하는 경우에는 OR (|) 연산을 안 함.

PINB (Port B Input)

Bit	7	6	5	4	3	2	1	0
0x16 (0x36)	PINB7	PINB6	PINB5	PINB4	PINB3	PINB2	PINB1	PINB0
Read/Write	R/W	R/W	R/W	R/W	R/W	R/W	R/W	R/W
Initial Value	N/A	N/A	N/A	N/A	N/A	N/A	N/A	N/A

[특수 용도] Output Mode 인 경우
Bit를 1로 설정하면 그 핀의 출력 값이 반전 됨 (0→1, 1→0).

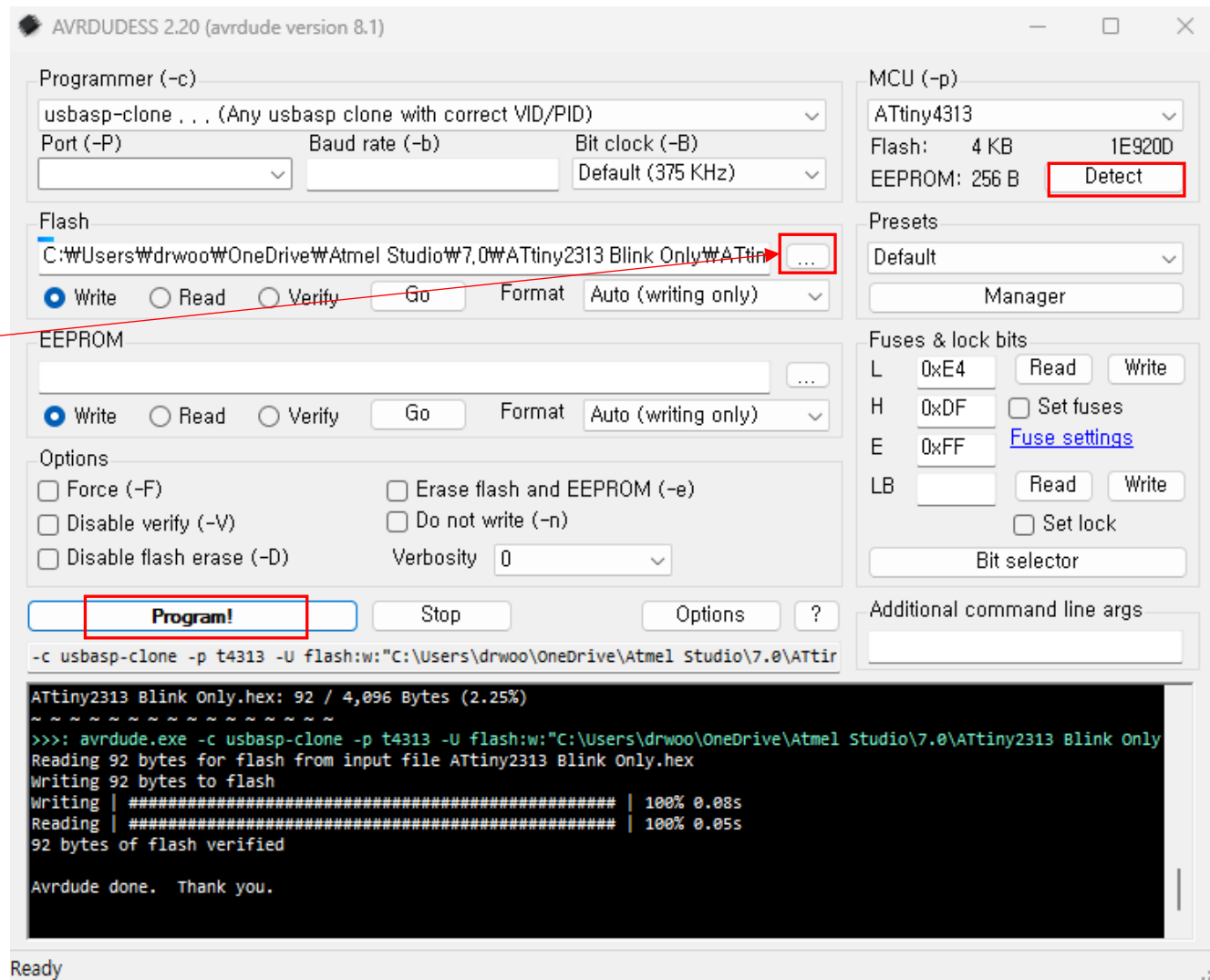
Build



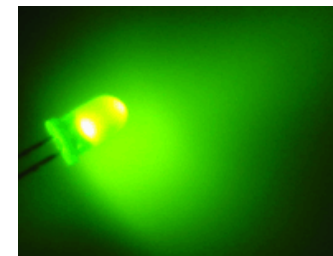
새로 생성된 기계어 파일 업로드

② 파일 경로 버튼을 눌러서 Blink.hex를 지정.

③ Program 버튼을 누르면 기계어 코드가 ATtiny로 업로드 됨.



① Detect 를 눌러서 ATtiny4313이 인식됨을 확인.



④ 코드 업로드가 끝나면 Blink 프로그램이 ATtiny에서 자동 실행됨.